ARIMA NOTEBOOKS

OPEN-SOURCE · LOCAL-FIRST · MULTI-LANGUAGE

PRODUCT BROCHURE · V3.2

Arima Notebooks

A modern, multi-language notebook for the agentic era — a **cross-platform execution plane** where people and AI agents collaborate on the same code. **Java · JShell · JavaScript · TypeScript · C# · F# · C++ · Python** — eight languages, every one first-class — plus **Agents**, a ninth kind of cell written in natural language and run by your agent CLI. Everything runs on your machine. No cloud account. No telemetry.

JShell · Java · JavaScript · TypeScript · C# · F# · C++ · Python · Agents

An open-source product by **Suresh Chande** — founding contributor

github.com/snchande/arima-notebooks

01 What is Arima Notebooks?

Arima Notebooks is part of the **Arima** platform — a **ground-up modernization of the notebook**, locally hosted and browser-based, that treats many rich languages as equals and is built for a world where humans and AI agents work on code together. Eight languages run side by side today — JavaScript, TypeScript, C#, F#, C++, Java, JShell, and Python, with **more to come** — each with real compilation, real dependencies, and real tooling. Alongside them sits a genuinely new unit of software: the **Agent** — a cell whose body is **natural language** rather than code, executed by a local **agent CLI**. Three AI co-pilots (Claude, GitHub Copilot, Antigravity) are wired in **locally**, and the **whole system is exposed via MCP** so any agent can drive it. The notebook becomes a **shared artifact** — you, the UI, an AI co-pilot, and any MCP agent are all first-class users of it. Everything runs on your machine. No cloud account. No telemetry.

The notebook, rethought. Two shifts left the classic notebook behind: real work is now **polyglot**, and code is a **team sport with AI**. Arima Notebooks keeps the best of the lineage — cells, kernels, rich output, pipelines — and reimagines it as a shared, cross-platform execution plane: **every language first-class**, **MCP exposing every capability**, and a workflow where you can *personalize Arima Notebooks to your needs in minutes.* Add a feature. Ask your CLI to package and open a PR. Ship it. *Tools should be shaped by the people who use them.*

Arima Notebooks is **additive, not adversarial**. It reads and writes `.ipynb` alongside its native `.vnb` format, so you can prototype here and export there freely — pick the canvas that gives you the best leverage.

What's inside this brochure

01 What is Arima Notebooks? 02
02 Capabilities & Features 03
03 Eight languages, one canvas 04
04 Architecture 05
05 Install & quick start 06
06 Your first notebook 07
07 AI assistance & the MCP server 08
08 Agents & Skills 09
09 Contributor playbook 10
10 Security & quality gates 11
11 About & contact 12

At a glance

- **8 languages** — JShell · Java · JS · TS · C# · F# · C++ · Python — one UI.
- **CLI inside** — Claude · Copilot · Antigravity, all local.
- **MCP outside** — Whole system exposed to any agent over MCP.
- **.ipynb interop** — Round-trip with Jupyter — pick your canvas.
- **Live output** — STOMP/WebSocket streaming. Zero polling.
- **3 package mgrs** — Maven · npm · NuGet — one click each.
- **Pipeline DSL** — `//@ anchor` / `//@ depends` across all 7 modes.
- **Personalize fast** — Add features in minutes — your CLI opens the PR.

02 Capabilities & Features

Every capability in Arima Notebooks is designed around two principles: *your environment, your machine*, and *fast iteration from idea to executed cell*. Here is the full feature surface.

JShell-native execution

Every JShell cell shares one persistent session per notebook. Variables, imports, and method definitions persist across cells exactly like a classic REPL.

Full `javac` compile mode

Switch any cell to Java mode to write a full `public class`. Compiled with `javax.tools`, line numbers normalised, stdout/stderr captured.

JavaScript & TypeScript

Node 18+ for JS; Node 22.6+ type-stripping for TS, with optional `tsc --noEmit` for full diagnostics. Both share the same npm modules.

C# & F# via .NET SDK

C# 9+ top-level programs via `dotnet run`; F# scripts via `dotnet fsi`. NuGet packages auto-injected as `<PackageReference>` or `#r "nuget:"`.

C++ with auto-detected toolchain

MSVC, GCC, or Clang — auto-detected on `PATH`. 25 standard headers pre-included. Notebook mode wraps cells in `main()` automatically.

Real-time output

STOMP over SockJS streams stdout/stderr live as the cell runs. No polling, no buffering surprises.

Python via `python3`

Cells run in a `python3` subprocess with `data/pypi-packages/` on `PYTHONPATH`. Anchors are injected as comments so pipelines work unchanged.

Four package managers

Maven (`groupId:artifactId:version` → JShell classpath), **npm** (JS + TS, `data/npm-modules/`), **NuGet** (C# `<PackageReference>` / F# `#r "nuget:"`), and **PyPI** (`pip --target` with a live install log). One-click install in every case.

Agents & Skills

Author an agent as a notebook, then list and run it from the **Agents** tab or over MCP (`barista_list_agents`, `barista_run_agent`). Agents compose into pipelines like any other cell — including multi-agent review chains.

Pipeline orchestration

Annotate cells with `//@ anchor` and `//@ depends`. The orchestration service builds a DAG, topologically sorts, and runs ancestors before your target cell. Works in all 8 languages, including cross-notebook references.

Multi-provider AI

Claude & Antigravity run as local CLI subprocesses; GitHub Copilot runs via the GitHub Copilot SDK (which drives the local `copilot` CLI). Switch providers from the AI tab. No API keys stored by Arima Notebooks; provider auth lives with the CLI.

AI language conversion

Switch a cell from Java to C#, or from JS to TypeScript, and Arima Notebooks offers to convert the existing code using the active AI provider. Same intent, new dialect.

MCP server

Expose Arima Notebooks itself as an MCP tool server — Claude Code, Claude Desktop, and custom agents can drive cells, manage packages, and read notebook state programmatically.

Variable inspector

Inspect live variable state across JShell, JS, TS, C#, F#, C++, and Python sessions. See types, values, and structure without printing.

Built-in data science stack

XChart, Apache Commons Math, Tablesaw, OpenCSV pre-loaded for Java. `simple-statistics` and `mathjs` for JS/TS. `barista.table()` / `baristaDisplay()` helpers across all runtimes.

39 built-in tutorials

Read-only curriculum from *JShell-101* through *Python-301*, plus a dedicated **Agents & Skills** track (*agent-101* → *agent-601*). Organised into per-language tabs; personal notebooks stay separate.

03 Eight languages, one canvas

Each cell carries a mode badge that mirrors the language colour shown in the live UI. Switch modes from the badge button at the top of any cell.

JShell — shared session

Every JShell cell in the notebook shares one `jdk.jshell` instance. Pre-loaded imports for `java.util`, `java.time`, XChart, Tablesaw, Commons Math, and the `BaristaDisplay` helpers. Variables and methods persist across cells until you press **Restart**.

Java — full classes

Switch a cell to Java mode for a complete `public class Main` with `main()`. Compiled via `javax.tools`, errors are line-number-normalised, stdout/stderr captured from the subprocess.

JavaScript — Node.js

Runs in a `node -e` subprocess. `require()` resolves from `data/npm-modules`. `barista.table()`, `barista.html()`, `barista.display()`, `barista.stats()` are available globally.

TypeScript — Node 22.6+

Cells run via Node.js's built-in `--experimental-strip-types`. Install `typescript` globally to enable `tsc --noEmit` diagnostics with proper line numbers. Same helpers as JS, full type signatures.

C# — `dotnet run`

Cells are wrapped as a C# 9+ top-level program inside a temp `.csproj`. NuGet packages auto-injected as `<PackageReference>`. Type declarations are moved to the end automatically to satisfy CS8803.

F# — `dotnet fsi`

Runs as an `.fsx` script. Inline `#r "nuget:"` directives are extracted and hoisted to the top of the file so `fsi` resolves them before compilation. `System`, `System.Linq`, and generics are pre-opened.

C++ — MSVC / GCC / Clang

Compiler auto-detected on `PATH` with `-std=c++17 -Wall`. 25 standard headers pre-included. Notebook mode wraps in `main()` automatically; if your cell declares `int main(` Arima Notebooks treats it as a complete program. Declarations are split to global scope, statements stay in `main()`. Helpers: `baristaHtml`, `baristaDisplay`, `baristaTable`.

Python — `python3`

Cells run in a `python3` subprocess with `data/pypi-packages/` on `PYTHONPATH`, so anything installed from PyPI imports normally. Install packages from the Packages tab (`pip --target` under the hood) and watch the live pip log. Pipeline anchors are injected as comments, so `#@ anchor` / `#@ depends` work exactly as in every other language.

Example — pipeline across cells

```
//@ anchor: load-data
//@ description: Load CSV from disk (JShell)
var data = Table.read().csv("data.csv");

//@ anchor: clean-data
//@ depends: load-data
var clean = data.dropRowsWithMissingValues();

//@ anchor: visualize
//@ depends: clean-data
Plot.show(clean.column("sales").asDoubleArray());
```

04 Architecture at a glance

Arima Notebooks is a single Spring Boot 3 application on Java 21. The browser loads static HTML/CSS/JS directly from Spring's static resource handler — there is no frontend build step. REST and STOMP endpoints sit in front of a fleet of execution services, each owning one language runtime.

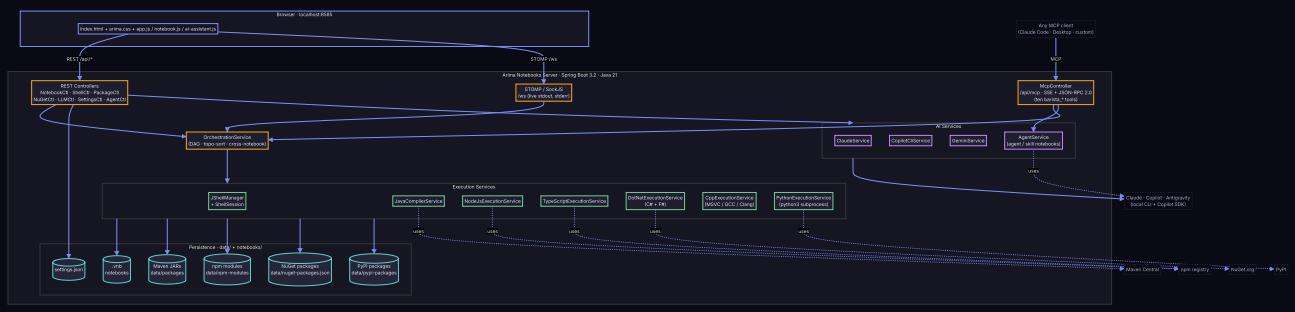

Figure 1 — Component map. Browser ↔ Spring Boot ↔ language-specific execution services ↔ local disk & package registries.

Execution flow for a single cell

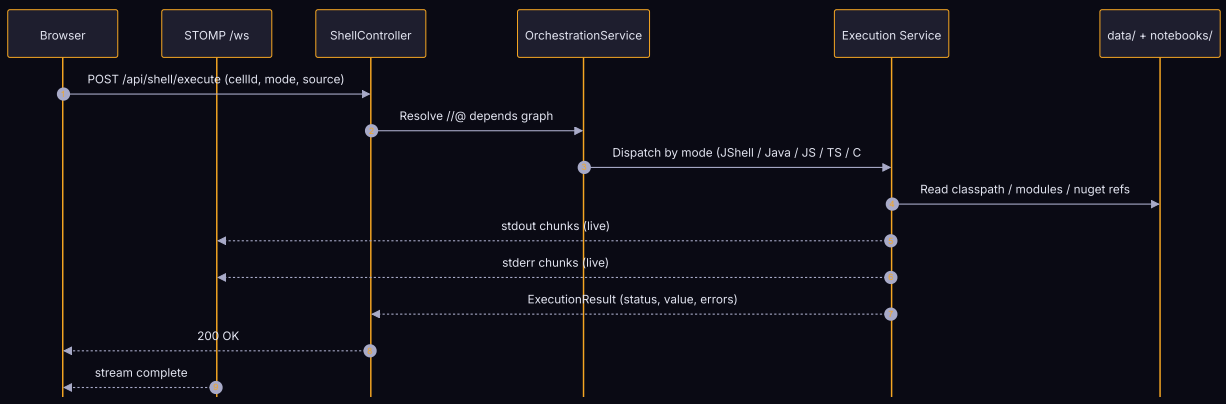

Figure 2 — One cell, end-to-end. Real-time output streams over STOMP while the REST request stays open for the final result.

05 Install & quick start

One JAR, one config file, one port. The `arima` launcher detects what is missing, builds the JAR if needed, starts the server, and opens your browser.

Prerequisites

Tool	Version	Why
JDK	17+ (21 recommended)	JShell + Spring Boot runtime. Must be a JDK, not just JRE.
Maven	3.8+	Build from source. Skip if you only run a published JAR.
Node.js	18+ (22.6+ for TS)	JavaScript / TypeScript cells + npm packages.
.NET SDK	6.0+	C# (`dotnet run`) and F# (`dotnet fsi`).
C++ compiler	any	MSVC on Windows, GCC or Clang on Linux/macOS — auto-detected.
Python	3.9+	Python cells + PyPI packages. Without it, the other seven languages work normally.
AI CLI	latest	Optional. Claude CLI / GitHub Copilot CLI (SDK) / Antigravity CLI (`agy`).

Five-step quick start

1 Clone the repo
`git clone https://github.com/snchande/arima-notebooks.git && cd arima-notebooks`

2 Start with the arima CLI
Windows CMD: `arima` · PowerShell: `./arima.ps1` · Linux/macOS: `./arima.sh`

3 Open the browser
Opens `http://localhost:8585` automatically. The first build downloads Maven deps.

4 Create your first notebook
Folder icon → **+ New Notebook**. A starter code cell opens immediately.

5 (Optional) Enable AI
Install `claude` , `copilot` , or `agy` ; pick it in **Settings → AI Provider**. No API keys touch Arima Notebooks.

Running the JAR directly

```
java --add-opens=jdk.jshell/jdk.jshell=ALL-UNNAMED \
     --add-opens=java.base/java.lang=ALL-UNNAMED \
     --add-exports=jdk.jshell/jdk.jshell=ALL-UNNAMED \
     -jar target/arima-notebooks-1.0.0-SNAPSHOT.jar
```

Tip. If PowerShell blocks `./arima.ps1` , run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` .

06 Your first notebook

Arima Notebooks opens with one empty JShell cell. From here you can switch modes, install packages, chain dependencies, and ask an AI to fill in the body.

JShell mode — shared state across cells

```
// Cell 1 (JShell)
var name = "Arima";
var version = 1.2;

// Cell 2 (JShell) — name and version are already in scope
System.out.printf("Hello from %s v%.1f%n", name, version);
```

Java mode — full class compilation

```
public class Fibonacci {
  public static void main(String[] args) {
    int a = 0, b = 1;
    for (int i = 0; i < 10; i++) {
      System.out.print(a + " ");
      int t = a + b; a = b; b = t;
    }
  }
}
```

Installing a Maven package

Packages tab → `com.google.code.gson:gson:2.10.1` → **Install**. The JAR lands on the active JShell classpath in seconds — no restart.

Pipeline orchestration — cells with dependencies

```
//@ anchor: parse
//@ description: Parse the JSON input
var json = "{\"x\": 42}";
var obj  = new Gson().fromJson(json, Map.class);

//@ anchor: render
//@ depends: parse
barista.display(obj);
```

Run with Dependencies on *render* runs *parse* first, in topological order. The same syntax works in Java, JS, TS, C#, F#, C++, and Python.

Keyboard shortcuts

`Ctrl+Enter`	Run current cell
`Shift+Enter`	Run cell and move focus to next
`Ctrl+\`	Toggle AI panel
`Ctrl+S`	Save notebook
`Ctrl+Shift+P`	Command palette

07 The Agentic Era — CLI inside, MCP outside

Arima Notebooks treats AI as **first-class infrastructure**, not a side panel. Three things make this real: a CLI runs *inside* every notebook session as a local subprocess; the entire Arima Notebooks capability surface is *outside*-exposed over MCP so any agent can drive it; and you can personalize Arima Notebooks itself the same way — by asking the same CLI to write the change and open the PR. No API keys live with Arima Notebooks. Authentication stays in the CLI you trust.

Claude
Install: claude.ai/code → run `claude auth`. Used via the `claude` binary on PATH.

GitHub Copilot
Install the `copilot` CLI (v1.0.55-5+) and authenticate it — the GitHub Copilot SDK drives it.

Antigravity
Install `agy` (antigravity.google/docs/cli-install) → run `agy` to sign in.

What you can ask the assistant

- *"Write a C++ cell that sorts a vector using std::sort and prints the result."*
- *"Explain what this cell does"* — attach a cell with the 🤖 button.
- *"Convert this Java cell to C#."* — Arima Notebooks offers conversion automatically when you switch a cell's mode.
- *"Generate a notebook showing Java streams with examples."* — produces a full `.vnb` file.
- *"Why did this cell fail?"* — the failed cell's stderr is attached automatically.

Arima Notebooks as an MCP server — the whole system, exposed

Pick the canvas that fits the moment. Drive Arima Notebooks from your terminal, from Claude Code or Claude Desktop, from a custom agent — or from Arima Notebooks itself. Same capability surface, three ways in.

Ten tools over JSON-RPC 2.0, at `/api/mcp/sse` + `/api/mcp/messages`:

`barista_execute_code`	Run code in any of the eight modes; returns the full execution result.
`barista_list_notebooks` `barista_read_notebook`	Enumerate `.vnb` files including tutorials, and read one back.
`barista_create_notebook` `barista_append_cell`	Create notebooks and append cells programmatically.
`barista_search_cells`	Search across every cell in every notebook.
`barista_run_pipeline`	Execute an anchor DAG, ancestors first.
`barista_load_module`	Install Maven, npm, NuGet, or PyPI packages on behalf of an agent.
`barista_list_agents` `barista_run_agent`	Discover the agents defined in this workspace and run one against a task.

Personalize Arima Notebooks in minutes. If a capability is missing, the workflow is the point: ask your CLI to add the feature — service, controller, UI hook — verify with `arima rebuild`, and ask the CLI again to package the change as a PR. New capability in less than an hour. If it helps you, it probably helps others; ship it back.

08 Agents & Skills

An agent is **a new kind of software entity** — and the first one in Arima Notebooks whose logic is not code. A function is written in syntax and executed by a runtime. An agent is **written in natural language** and executed by an **agent CLI**: you state the intent, and Claude, Copilot, or Antigravity carries it out. Same call sites, same pipelines, same MCP surface as everything else — but the body is prose, not a language.

Concretely, an agent is a notebook: any `.vnb` with `metadata.kind = "agent"` (or `"skill"`) becomes a callable unit that cells, pipelines, and external MCP clients can invoke — authored, versioned, and shared like any other notebook.

The agent cell

Inside a normal notebook, an *agent cell* names the agent to call and the task to give it. Anchors from the surrounding notebook are interpolated with `{{…}}`, so an agent can read the live output of the cells above it:

```
//@ agent: code-reviewer
//@ bind: review
//@ depends: build-report
Review the following output for correctness and style.
Flag anything that would fail CI:

{{build-report}}
```

- **`//@ agent:`** — Which agent notebook to dispatch to. Resolved by id against the agents in your workspace.
- **`//@ bind:`** — Binds the agent's answer into a JShell variable, so downstream cells can use the result as data rather than prose.
- **`{{anchor}}`** — Interpolates the current output of any anchored cell into the task text — the agent sees real results, not a stale copy.
- **`//@ depends:`** — The same orchestration DSL as every other cell. Agents are ordinary nodes in the DAG and run in dependency order.

Three ways to run one

Agents tab	Browse every agent and skill in the workspace, give it a task, and watch the run stream live.
In a pipeline	An agent cell is a DAG node — chain several to get multi-agent review, where one agent grades another's output.
Over MCP	`barista_list_agents` discovers them; `barista_run_agent` runs one. External agents can call your agents.

Runs stream over the notebook's existing STOMP `partial_output` channel, so output appears token-by-token with no extra endpoints. Providers are pluggable behind an `AgentProvider` interface — Claude ships today; another is a single new bean.

Learn it by running it. The tutorial library ships a full **Agents & Skills** track — *agent-101* (Explain Code) through *agent-601* (MCP-driven Agent), taking in a code reviewer, a test writer, a reviewer inside a pipeline, and a multi-agent review chain along the way.

09 Personalize & contribute

Arima Notebooks is meant to be **shaped by the people who use it**. Clone, describe the change to your AI CLI, verify, ask the same CLI to package the PR. Instruction files for Claude, Copilot, and Antigravity keep the agent inside the architecture.

1 — Fork & clone

```
git clone https://github.com/<your-fork>/arima-notebooks.git
cd arima-notebooks
git remote add upstream https://github.com/snchande/arima-notebooks.git
git checkout -b feat/<short-name>
```

2 — Pick your AI co-pilot

Claude CLI

Run `claude` in the repo root. It reads `CLAUDE.md` and `AGENTS.md` automatically — both encode the architecture rules.

GitHub Copilot

Inline in VS Code or via `gh copilot`. Reads `.github/copilot-instructions.md` — same architecture rules.

Antigravity CLI

Run `agy` in the repo root. Reads `AGENTS.md` (and `GEMINI.md`) — same architecture rules.

3 — Describe the feature, let the CLI write it

Describe the capability you want. The agent files keep it inside the layered architecture — *model → service → controller → static UI*. No layer-merging, no surprise network calls.

4 — Verify locally

```
arima rebuild
arima start
# exercise your feature in the browser; check WebSocket output
mvn test
pwsh ./scripts/security-check.ps1   # or .sh on Linux/macOS
```

5 — Ask your CLI to package and PR

One more prompt: *"package these changes as a PR against `master`, fill the PR template, attach screenshots if UI."* If it helps others, ship it back.

6 — Wait for the green checks

Every PR triggers GitHub Actions checks before review:

`build`	Maven compile + unit tests on JDK 21.
`security-scan`	Runs `scripts/security-check.ps1` / `.sh` — see page 11.
`architecture-lint`	Verifies you didn't break layer boundaries the agent files describe.
Founder review	Required code-owner review by Suresh Chande before merge to `master`.

The compact. Use AI freely. Read `AGENTS.md` first. Stay inside the layer. Update the docs. Green checks before review — the faster they pass, the faster your PR moves. *Tools should evolve at the speed of the people using them.*

10 Security & quality gates

Every pull request runs an automated security check before it can be marked **ready for review** — catch obvious risk before a human spends time on it.

The PR security pipeline

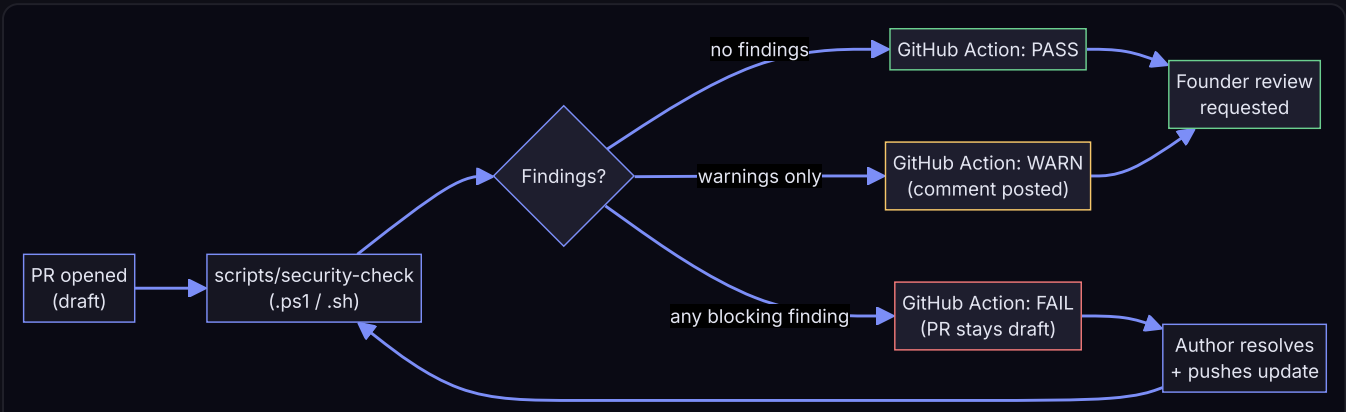

Figure 3 — A PR cannot become "ready for review" until `security-check` reports zero blocking findings.

What the script checks

Check	Why it matters
Secret patterns (`sk-…`, AWS keys, private-key PEM headers, GitHub tokens)	Prevents accidental credential leaks. Blocks the PR.
Hard-coded API keys in `data/settings.json`	The settings file is `.gitignore`-d for a reason. Blocks.
Calls to `Runtime.exec` with user-controlled strings	Command-injection risk in execution services. Blocks.
New outbound HTTP hosts (beyond Maven Central, npm, NuGet, PyPI, AI CLIs)	Preserves the local-first guarantee. Blocks unless allow-listed.
Dependency vulnerabilities (`mvn dependency:check`)	Known-CVE detection. High-severity blocks; medium warns.
Architecture lint — layer crossings (frontend → DB, controller → controller)	Preserves the contract in `AGENTS.md`. Blocks.
Test coverage for changed files (best-effort)	Warns if a new service has no companion test.

Local pre-flight

```
# Windows
pwsh ./scripts/security-check.ps1

# Linux / macOS
./scripts/security-check.sh
```

Run before you push — the local script is identical to CI's. **If it fails locally, it fails in CI.**

Founding contributor review. Even after green checks, every change to `master` requires a code-owner review by **Suresh Chande**. CODEOWNERS is wired so GitHub blocks merge automatically.

11 About & contact

Arima Notebooks is an open-source product distributed under the **MIT License**. It is built and maintained by **Suresh Chande** with help from the community — and with a healthy population of AI co-pilots, which is exactly the workflow Arima Notebooks is built to support. This project stands on the shoulders of every great notebook tool that came before it; the goal is to *add* to that lineage, not replace it.

Suresh Chande

Founding contributor · Maintainer

✉ suresh.chande@gmail.com

↗ github.com/snchande/arima-notebooks

How to get involved

File an issue

Bugs, feature requests, tutorial ideas — anything. Use the issue templates under `.github/ISSUE_TEMPLATE`.

Open a discussion

Got an idea but not sure where it fits? Start a GitHub Discussion. The maintainer reads them all.

Send a PR

Follow the *Contributor Playbook* on page 10. Small, focused PRs are merged fastest.

Share a tutorial

Ship a `.vnb` file to the tutorial library. New languages, new techniques — all welcome.

Licence & attribution

Thanks. If Arima Notebooks saves you time, the simplest way to say thanks is a star on the repo and a PR — even a one-line typo fix lands you on the contributor list.